ANÁLISIS FORENSE Y ESTENOGRAFÍA

EXTRACCIÓN DE ARCHIVO OCULTO MEDIANTE ESTEGANOGRAFÍA Y COMANDOS EN IMAGEN JPEG

AUTOR: **CRISTIAN MONTOYA A.**

ESPECIALISTA EN CIBERSEGURIDAD Y CUMPLIMIENTO

Diciembre, 2025

cristian-montoya-cyber
Montoya-CyberSec

ÍNDICE **2**

1. IDENTIFICACIÓN DEL TIPO DE ARCHIVO **3**

2. CÁLCULO DEL HASH SHA-256 DEL ARCHIVO ORIGINAL **4**

3. EXTRACCIÓN DE METADATOS: AUTOR Y DISPOSITIVO DE CAPTURA **6**

4. LOCALIZACIÓN Y EXTRACCIÓN DE ARCHIVO OCULTO MEDIANTE ESTEGANOGRAFÍA **7**

5. CÁLCULO DEL HASH SHA-256 DEL ARCHIVO OCULTO EXTRAÍDO **10**

6. CONCLUSIÓN GENERAL **11**

1. Identificación del tipo de archivo

Al momento de copiar el archivo **Actividad1** a la máquina virtual, Kali Linux, este sistema operativo lo reconoce como archivo de extensión **.jpeg** una vez se termina de pasar. Pero para corroborar este formato de archivo se coloca en la consola de Kali y se ejecuta el comando **file** sobre el archivo, con el fin de identificar el formato a partir de sus cabeceras internas y firma del contenedor.

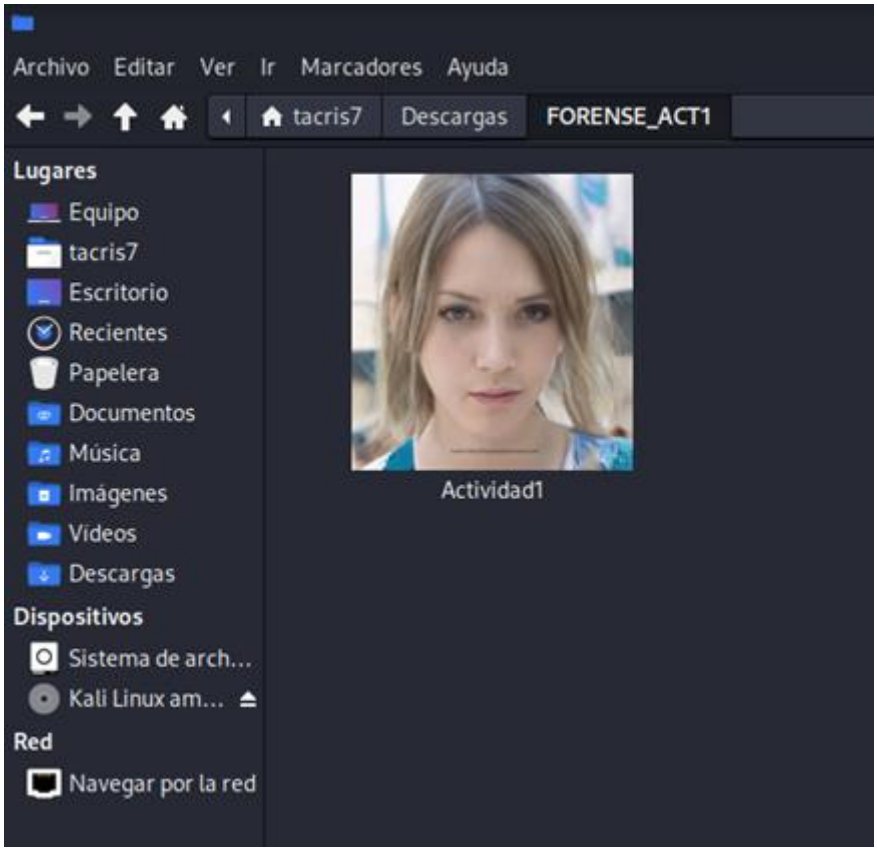

Figura 1. *Archivo Actividad1 detectado automáticamente como .jpeg*

Nota. Captura de pantalla de la *VM*

```
┌──(tacris7㉿Tacris7)-[~/Descargas/FORENSE_ACT1]
└─$ file Actividad1
Actividad1: JPEG image data, JFIF standard 1.01, resolution (DPI), density 96×96, segment length 16, Exif Standard: [TIFF image
data, big-endian, direntries=16, description=Chica, manufacturer=Sony, model=DSC-H300], baseline, precision 8, 883×935, componen
ts 3
```

Figura 2. *Identificación del tipo de archivo mediante el comando file en la VM*

Nota. Captura de pantalla obtenida desde la consola de *Kali Linux.*

Como resultado se obtiene que el archivo **Actividad1** corresponde a una imagen en formato **JPEG (estándar JFIF 1.01)**. Además, incorpora metadatos **EXIF** (estructura

TIFF interna usada por EXIF). La imagen es con profundidad de 8 bits (**precision 8**) y dimensiones **883 × 935 píxeles**, con densidad **96 × 96 DPI.**

Como segunda verificación, se inspeccionó el archivo en Windows a través de la herramienta de editor hexadecimal **HxD** aplicando el criterio de análisis de **magic numbers**.

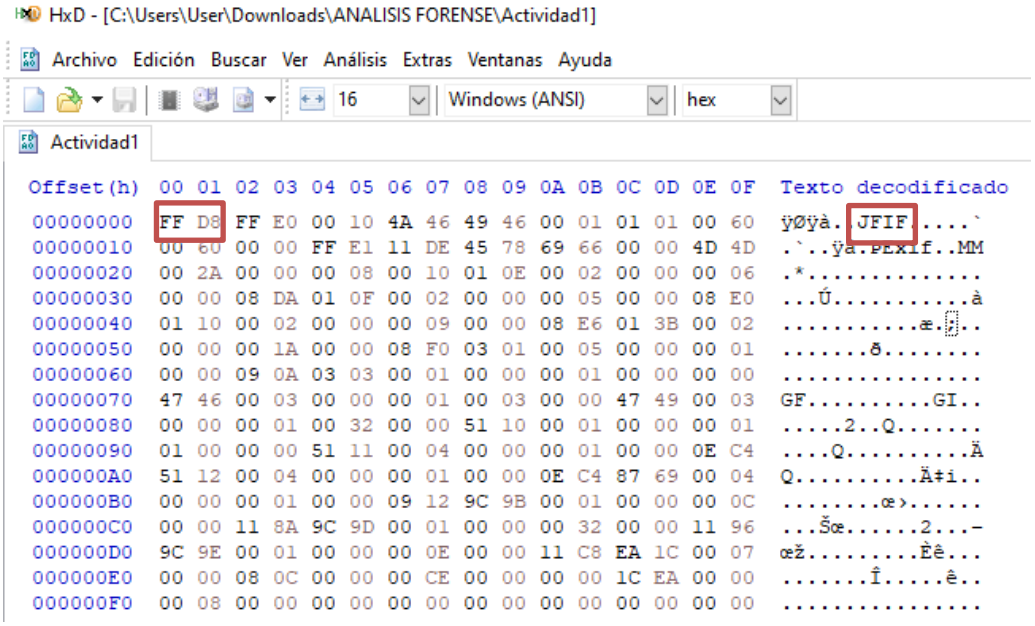

Figura 3. *Verificación de firma JPEG mediante editor hexadecimal HxD*

Nota. Captura de pantalla obtenida desde *HxD*

El archivo presenta una cabecera característica del formato **JPEG**, identificable mediante los magic numbers **FF D8**. Asimismo, se observan los identificadores JFIF y EXIF, lo que confirma con certeza que se trata de una imagen **JPEG** conforme al estándar JFIF, independientemente de su extensión o del sistema operativo utilizado.

2. Cálculo del hash SHA-256 del archivo original

Para el cálculo del hash del archivo **Actividad1** utilizando el algoritmo SHA-256, se hace uso de la herramienta **HashMyFiles**, en el entorno Windows. Esta herramienta permite obtener los distintos valores hash de un archivo de manera rápida y fiable.

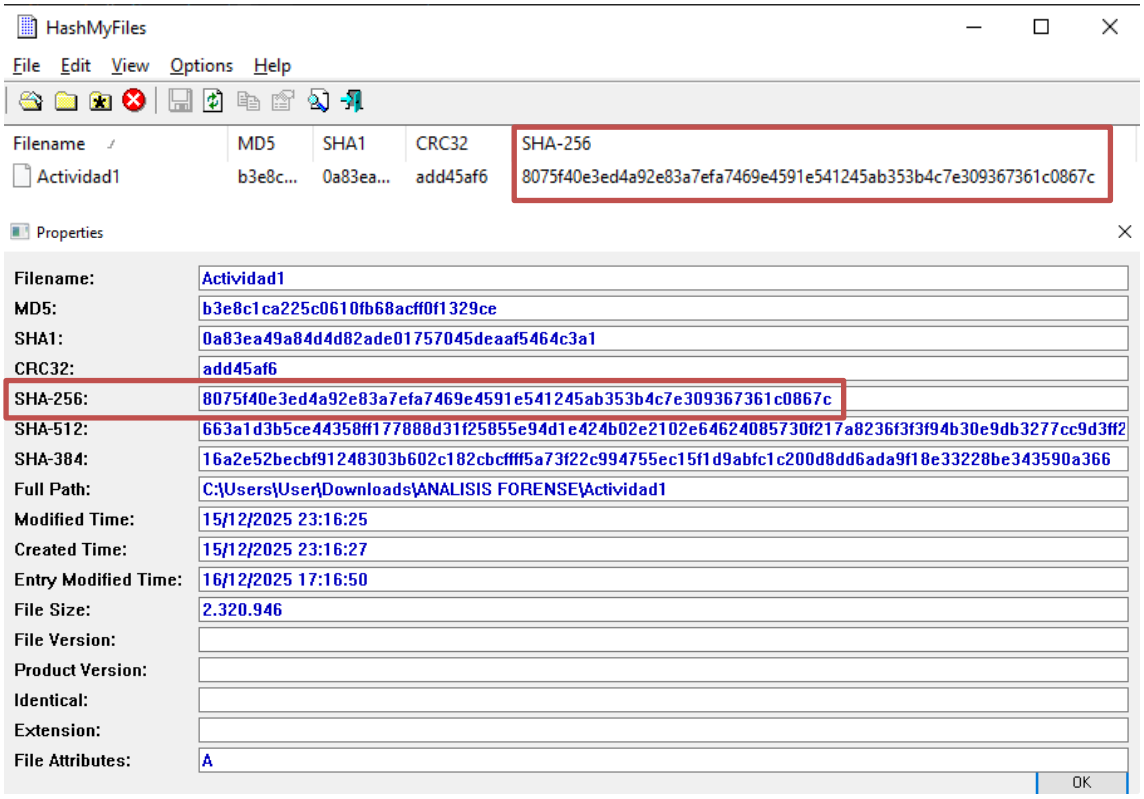

Figura 4. *Cálculo del hash SHA-256 del archivo Actividad1 mediante HashMyFiles.*

Nota. Captura de pantalla de la herramienta *HashMyFiles* en *Windows*.

Como resultado del análisis, se obtuvo el siguiente valor hash **SHA-256**:

8075f40e3ed4a92e83a7efa7469e4591e541245ab353b4c7e309367361c0867

Como verificación adicional, se realizó el cálculo del hash SHA-256 mediante PowerShell, utilizando el comando **Get-FileHash,** obteniéndose el mismo valor hash que el calculado previamente, lo que confirma la consistencia del resultado. Para ello abrimos PowerShell y nos dirigimos a la ruta donde esta el archivo, una vez dentro, ejecutamos el siguiente comando:

```
Get-FileHash -Algorithm sha256
```

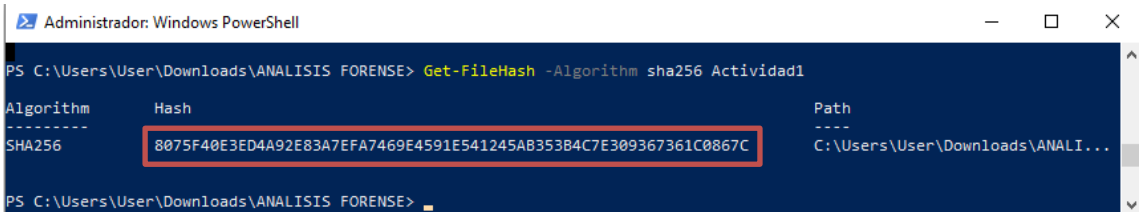

Figura 5. *Verificación del hash SHA-256 del archivo Actividad1 mediante PowerShell.*

Nota. Captura de pantalla de *PowerShell* en *Windows.*

3. Extracción de metadatos: autor y dispositivo de captura

En este apartado, se lo puede realizar tanto en Windows como en Linux haciendo uso de la herramienta **exiftool**, la cual permite extraer y visualizar los metadatos **EXIF/XMP** del archivo en cuestión. En este caso, este procedimiento se lo lleva a cabo en Kali Linux ya que esta herramienta se encuentra preinstalada en este entorno. Para ello introducimos el comando

```
exitfool Actividad1
```

Se nos muestra una serie de parámetros clave, entre ellos, tipo de archivo fecha de creación, autor, modelo de cámara, nombre del creador, ubicación geográfica y otros metadatos relevantes, como el título, descripción e información de los píxeles.

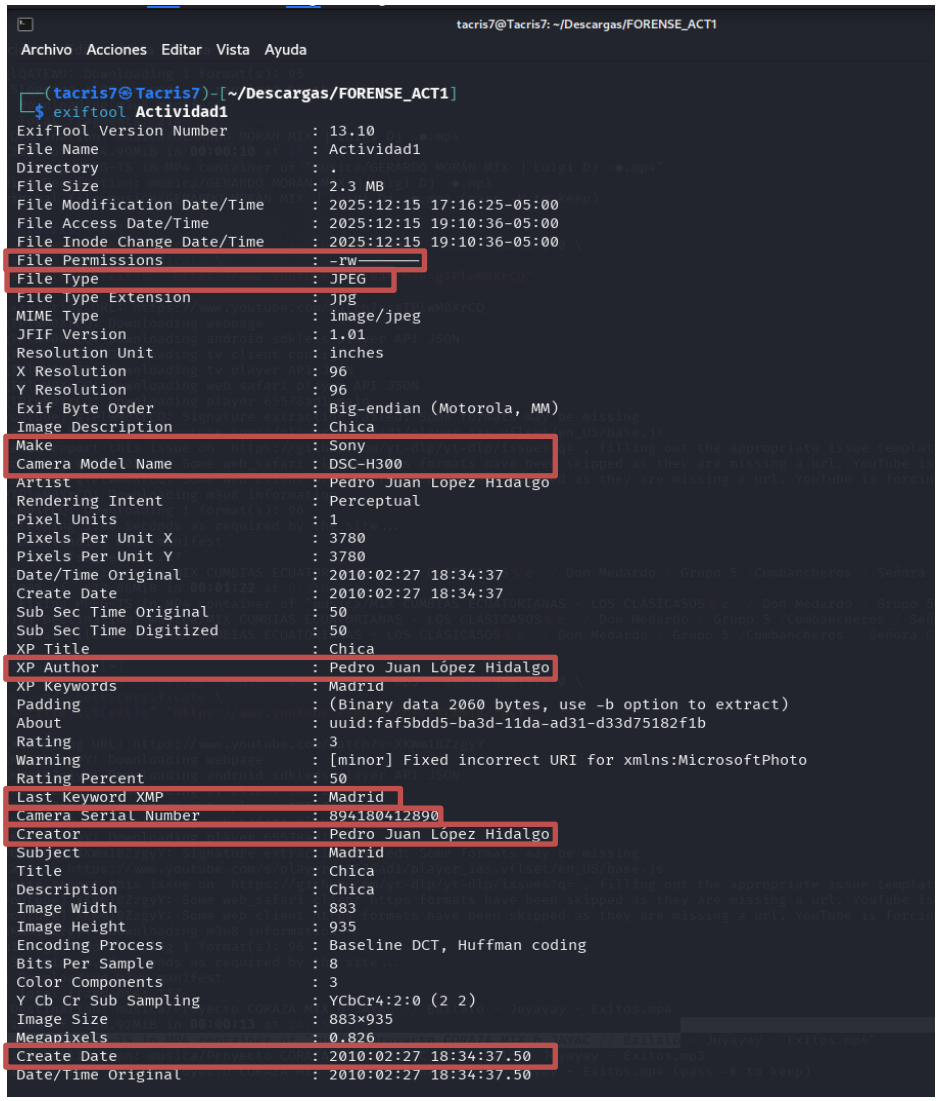

Figura 6. *Extracción de metadatos EXIF/XMP del archivo Actividad1 mediante ExifTool.*

Nota. Captura de pantalla obtenida desde la consola de *Kali Linux.*

A partir de la información extraída, se identifican campos relevantes que permiten determinar el autor del archivo y el dispositivo utilizado. Los campos **Artist, Creator** y **XP Author** indican como autor a **Pedro J. López H.** [Nombre Ficticio]**.** Por otro lado, los campos **Make** y **Camera Model Name** señalan que el archivo fue generado mediante una cámara digital **Sony**, **modelo DSC-H300.**

4. Localización y extracción de archivo oculto mediante esteganografía

Como primer paso, se evaluó la posible existencia de información oculta mediante técnicas clásicas de esteganografía, utilizando la herramienta **steghide**, la cual permite detectar y extraer datos ocultos en imágenes cuando estos han sido incrustados mediante esquemas de esteganografía. Para ello, se ejecutó el siguiente comando:

```
steghide info Actividad1
```

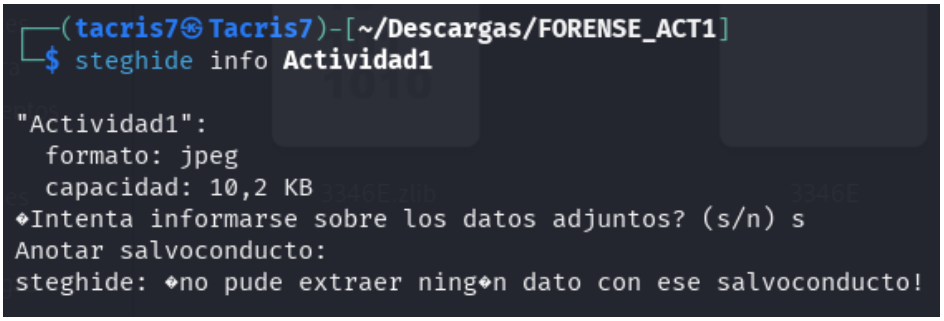

Figura 7. *Análisis inicial del archivo Actividad1 mediante la herramienta steghide.*

Nota. Captura de pantalla obtenida desde la consola de *Kali Linux.*

El resultado indica que el archivo tiene formato JPEG y muestra una capacidad estimada para ocultar datos. No obstante, al intentar acceder a la información adjunta, la herramienta solicita un “salvoconducto” (contraseña), y al no proporcionarse uno válido, no es posible extraer ningún dato. Esto sugiere que el contenido oculto no fue incrustado mediante esteganografía clásica gestionada por **steghide**.

Dado que **steghide** no permitió acceder al contenido oculto, se procedió a realizar un análisis del archivo a nivel binario utilizando la herramienta **binwalk**, la cual permite identificar datos embebidos mediante el reconocimiento de firmas internas de archivos, sin depender de contraseñas.

```
┌──(tacris7㉿Tacris7)-[~/Descargas/FORENSE_ACT1]
└─$ binwalk Actividad1

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
0             0x0             JPEG image data, JFIF standard 1.01
30            0x1E            TIFF image data, big-endian, offset of first image directory: 8
209939        0x33413         PNG image, 914 x 928, 8-bit/color RGB, non-interlaced
210030        0x3346E         Zlib compressed data, compressed
```

Figura 8. *Detección de datos embebidos en el archivo Actividad1 mediante binwalk.*

Nota. Captura de pantalla obtenida desde la consola de *Kali Linux.*

El análisis reveló la presencia de un segundo fichero incrustado, identificado como una imagen PNG, la cual está localizada a partir del offset **209939**, así como datos adicionales comprimidos en formato **zlib**. Esto evidencia la existencia de información oculta dentro del archivo original mediante el análisis de su estructura interna.

Posterior a esto, se intentó realizar la extracción automática de los datos detectados utilizando la opción de extracción de **binwalk**

```
binwalk -e Actividad1
```

```
┌──(tacris7㉿Tacris7)-[~/Descargas/FORENSE_ACT1]
└─$ binwalk -e Actividad1

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
210030        0x3346E         Zlib compressed data, compressed

WARNING: One or more files failed to extract: either no utility was found or it's unimplemented
```

Figura 9. *Intento de extracción automática de datos embebidos mediante binwalk.*

Nota. Captura de pantalla obtenida desde la consola de *Kali Linux.*

Sin embargo, **binwalk** no pudo reconstruir automáticamente el fichero **PNG**, generando un aviso de extracción incompleta. Este comportamiento es sugiere que los datos incrustados no siguen una estructura estándar completamente soportada por la herramienta.

Ante esta dificultad, se procedió a recuperar el fichero oculto mediante una extracción manual por **offset**, para la recuperación de datos embebidos. Para ello, se utilizó el siguiente comando **dd**, copiando los datos byte a byte a partir del offset previamente identificado:

```
dd if=Actividad1 of=fichero_oculto.png bs=1
```

Donde:

- **`dd`**: realiza una copia datos **byte a byte**
- **`if (input file)`**: archivo de entrada, en este caso el archivo **Actividad1**
- **`of (output file)`**: archivo de salida, nombrado **fichero_oculto**
- **`bs (block size)`**: tamaño de bloque, en este caso será **1 byte** a la vez
- **`skip`**: número de bytes que se omiten al inicio, en este caso, **209939** ya que desde este byte comienza la cabecera del fichero **PNG**.

```
┌──(tacris7㉿Tacris7)-[~/Descargas/FORENSE_ACT1]
└─$ dd if=Actividad1 of=fichero_oculto.png bs=1 skip=209939
2111007+0 records in
2111007+0 records out
2111007 bytes (2,1 MB, 2,0 MiB) copied, 20,7295 s, 102 kB/s
```

Figura 10. *Resultado de la extracción manual del fichero oculto mediante el comando dd.*

Nota. Captura de pantalla obtenida desde la consola de *Kali Linux.*

Se evidencia que se realizó con éxito la copia de **2,1 MB** de datos desde el archivo original, lo que indica que la extracción se realizó correctamente. Como resultado de este proceso, se genera el archivo **fichero_oculto.png**, el cual corresponde al fichero oculto dentro de la imagen original.

Posteriormente, se verificó de manera visual la recuperación del fichero oculto accediendo al directorio de trabajo, donde se observa la presencia del archivo **fichero_oculto.png** junto al archivo original **Actividad1**.

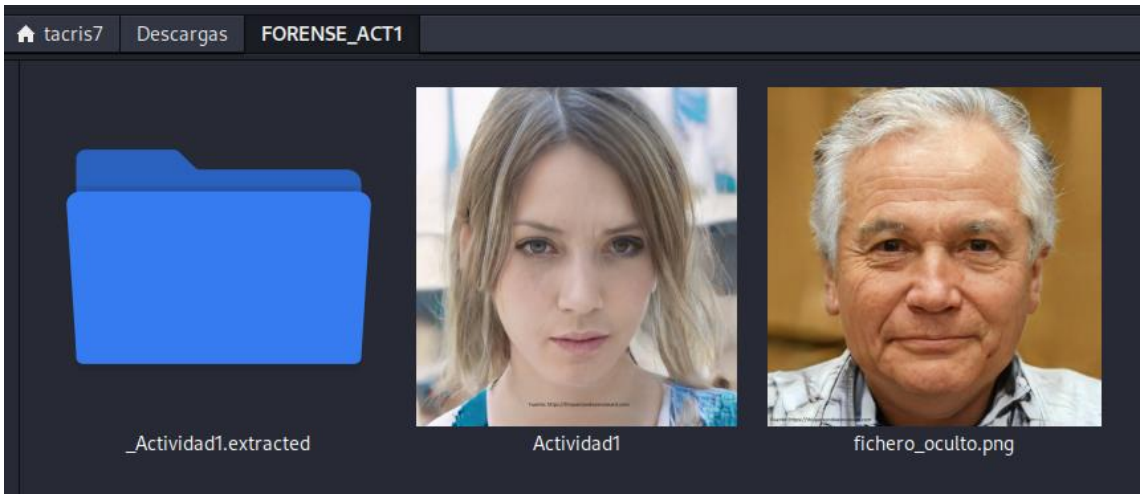

Figura 11. *Visualización del archivo original y del fichero oculto extraído*

Nota. Captura de pantalla obtenida desde la consola de *Kali Linux.*

Para verificar la correcta extracción del fichero oculto, se utilizó el comando **file**, confirmándose que el archivo recuperado corresponde a una imagen **PNG** con las mismas características detectadas previamente por binwalk.

```
┌──(tacris7㉿Tacris7)-[~/Descargas/FORENSE_ACT1]
└─$ file fichero_oculto.png
fichero_oculto.png: PNG image data, 914 x 928, 8-bit/color RGB, non-interlaced
```

Figura 12. *Verificación del tipo de archivo del fichero oculto extraído*

Nota. Captura de pantalla obtenida desde la consola de *Kali Linux.*

5. Cálculo del hash SHA-256 del archivo oculto extraído

Para este caso seguimos trabajando en Kali Linux. Una vez extraído el fichero oculto, se procedió al cálculo de su hash criptográfico utilizando el algoritmo SHA-256, con el fin de garantizar la integridad del archivo recuperado. El valor obtenido permite verificar que el fichero ha sido extraído de forma completa y que no ha sufrido modificaciones durante el proceso de análisis.

Para el cálculo del hash se utilizó el comando **sha256sum** en el entorno Kali Linux, aplicado directamente sobre el fichero extraído.

```
sha256sum fichero_oculto.png
```

```
┌──(tacris7㉿Tacris7)-[~/Descargas/FORENSE_ACT1]
└─$ sha256sum fichero_oculto.png

4fec65bc5eefb3c7f1258fbcff214b34670c9fe013b7b26ebe91e6db84626eef  fichero_oculto.png
```

Figura 13. *Cálculo del hash SHA-256 del fichero oculto extraído.*

Nota. Captura de pantalla obtenida desde la consola de *Kali Linux.*

Como resultado del cálculo, se obtuvo el siguiente valor hash SHA-256:

4fec65bc5eefb3c7f1258fbcff214b34670c9fe013b7b26ebe91e6db84626eef

6. Conclusión General

En esta actividad se aplicaron técnicas de análisis forense para identificar y analizar un archivo digital, determinándose de forma concluyente que el fichero Actividad1 corresponde a una imagen JPEG (JFIF). Asimismo, mediante el cálculo de hashes SHA-256, se garantizó la integridad tanto del archivo original como del fichero oculto extraído, permitiendo la verificación de los resultados obtenidos. El análisis de metadatos permitió identificar como autor del archivo a Pedro J. López H., así como el dispositivo de captura utilizado, correspondiente a una cámara digital Sony DSC-H300. Adicionalmente, se detectó la presencia de un fichero oculto incrustado dentro de la imagen, el cual fue localizado mediante análisis binario con binwalk y recuperado correctamente mediante una extracción manual por offset, obteniéndose una imagen PNG sin alterar la evidencia original.